Regularization and Optimization
October 26th, 2026

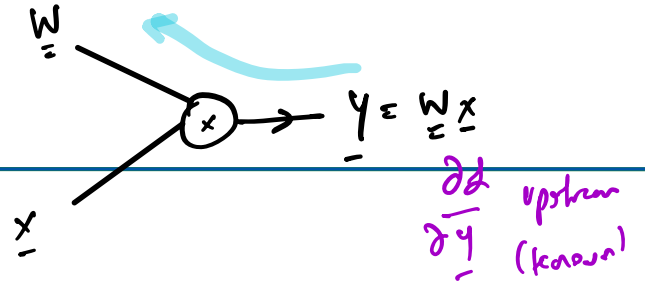
These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

Announcements

- HW #4 due ~~Friday October 30~~ Sunday Nov 1st, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late! Your responsibility to confirm upload of HWs**

- HW#1, 2, 3 solutions posted. → will be posted
- **Midterm 4th November 2026** (MSOL Midterm November 7th, Saturday); allowed one sheet (letter size), no devices or phones allowed in exam. We will have 2 rooms for the exams and will announce who will go where.
- Practice midterms will be posted soon, and solutions posted a week later. solutions → end of this week
- **Please read the rules handout carefully including for late HWs etc.**
- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.
- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available. **Midterm review on Friday, October 30th.**
- For midterm **CAE approved accommodation students have to contact CAE office to proctor exam** as we cannot do this ourselves. Please do this pro-actively.

Review of previous lecture

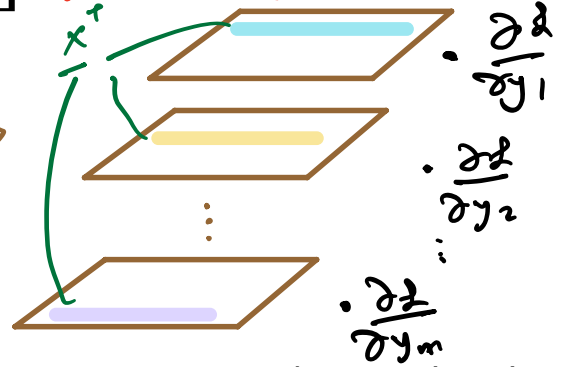


- **Multivariate chain rule and tensor derivatives**

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} \Longrightarrow y_k = \sum_{j=1}^{n} \boldsymbol{W}_{kj} x_j$$

Calculate $\dfrac{\partial y_k}{\partial \boldsymbol{W}_{i,p}} = \dfrac{\partial}{\partial \boldsymbol{W}_{i,p}} \left[\sum_{j=1}^{n} \boldsymbol{W}_{kj} x_j \right] = x_p \delta_{i=j} \delta_{k=p}$

$$\frac{\partial \mathscr{L}}{\partial \boldsymbol{W}} = \sum_{k=1}^{m} \frac{\partial y_k}{\partial \boldsymbol{W}} \frac{\partial \mathscr{L}}{\partial y_k} = \frac{\partial \mathscr{L}}{\partial \boldsymbol{y}} \boldsymbol{x}^\top$$



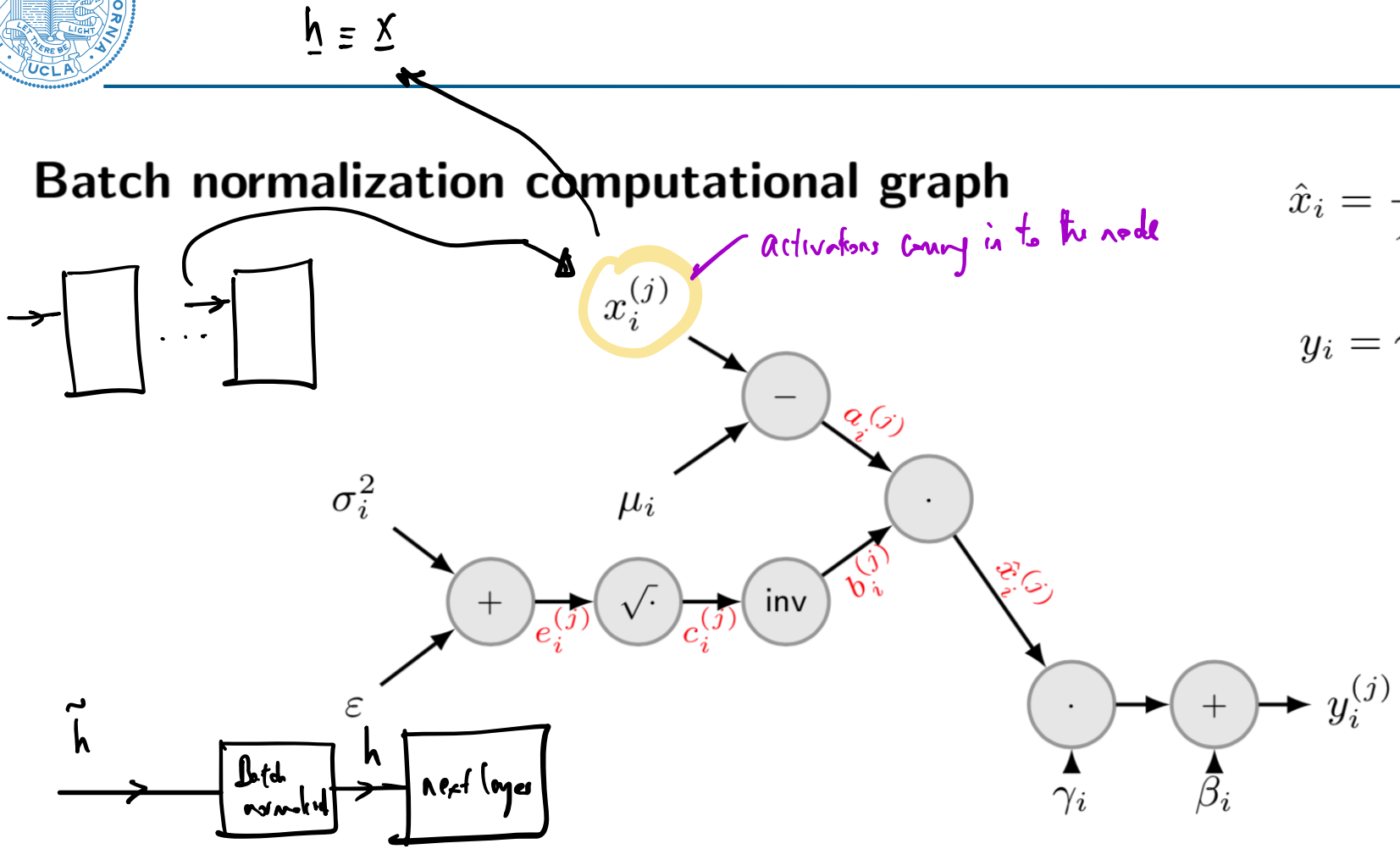
- **Initialization in neural networks**
  - Small weight initialization: gradients go to zero as the activations diminish
  - Large weight initialization: explosion in activations

Question: Intermediate initialization strategies?

$\mathbb{E}[h_{ij}] = \mathbb{E}[h_i] \;\forall j$ and $\mathrm{var}(h_{ij}) = \mathrm{var}(h_i) \;\forall j$

- **Xavier initialization**
  - Goal: equalize variance across layers in the initialization and set $\mathrm{var}(h_1) \approx \mathrm{var}(h_2) \approx \ldots \approx \mathrm{var}(h_L)$
  - $\mathrm{var}(w_{i,j}) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}} \Longrightarrow w_{i,j} \sim \mathcal{N}\left(0, \dfrac{2}{n_{\text{in}} + n_{\text{out}}}\right)$ for particular layer

Batch normalization



$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

batch mean

batch variance

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

learnable parameters

Convert mean to β_i and variance to γ_i^2

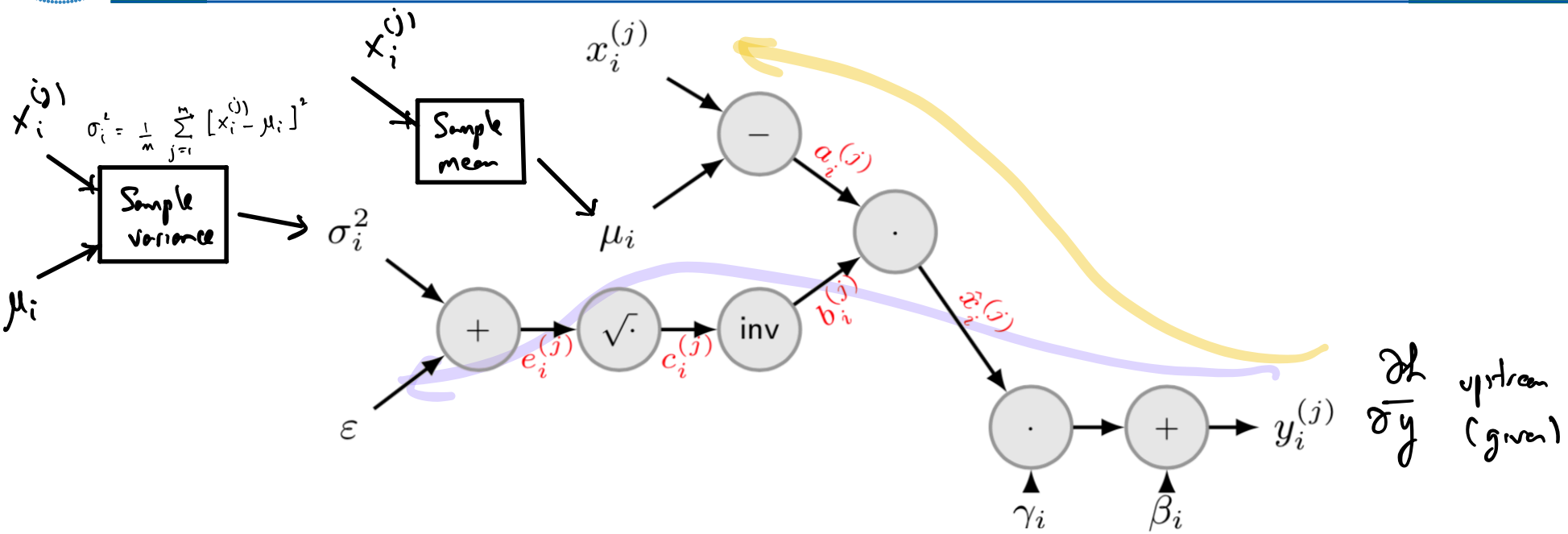
Gradients from backprop on next page.

We will drop the subscript i for convenience.

$$\mu_i = \frac{1}{m} \sum_{j=1}^{m} x_i^{(j)} \qquad m = \text{batch size}$$

$$\sigma_i^2 = \frac{1}{m} \sum_{j=1}^{m} \left[x_i^{(j)} - \mu_i \right]^2$$

Batch normalization



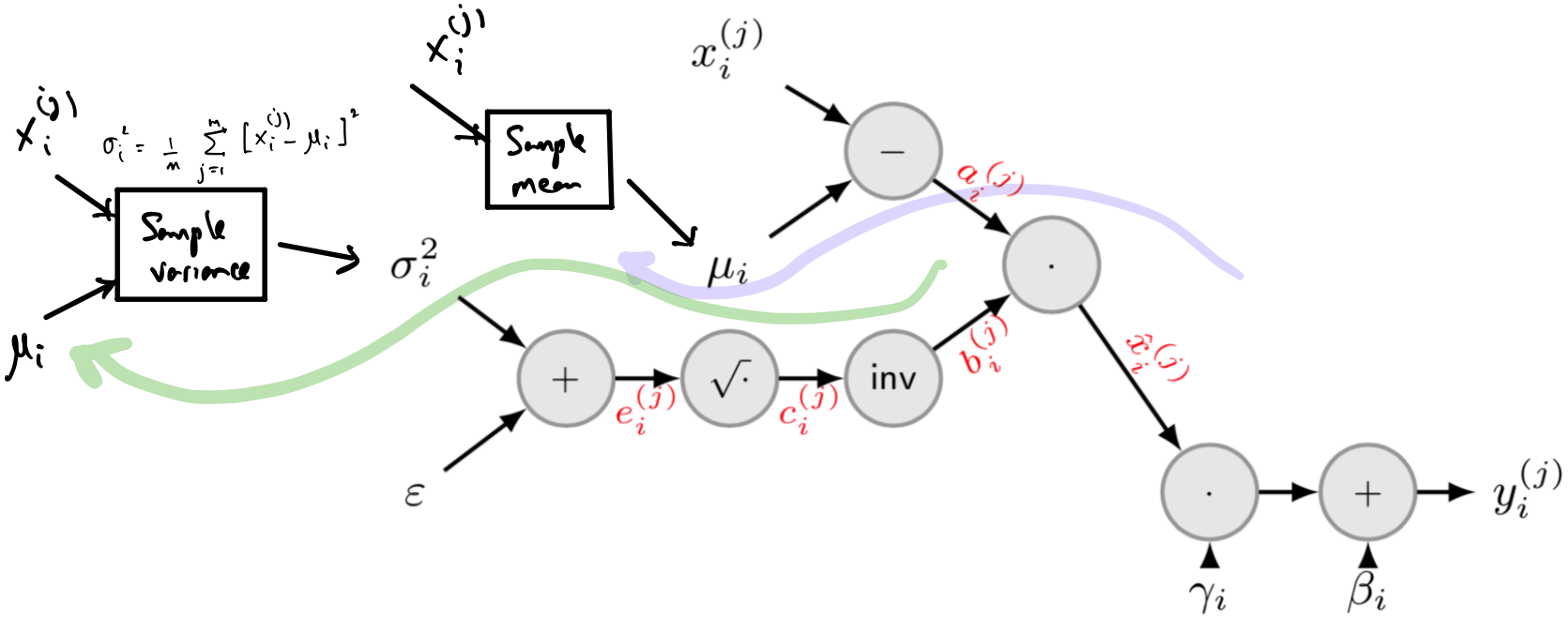
$$\frac{\partial \ell}{\partial \beta} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial y^{(j)}}$$

$$\frac{\partial \ell}{\partial \gamma} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial y^{(j)}} \hat{x}^{(j)}$$

$$\frac{\partial \ell}{\partial \hat{x}^{(j)}} = \frac{\partial \ell}{\partial y^{(j)}} \gamma$$

$$\frac{\partial \ell}{\partial a^{(j)}} = \frac{1}{\sqrt{\sigma^2 + \epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

Batch normalization



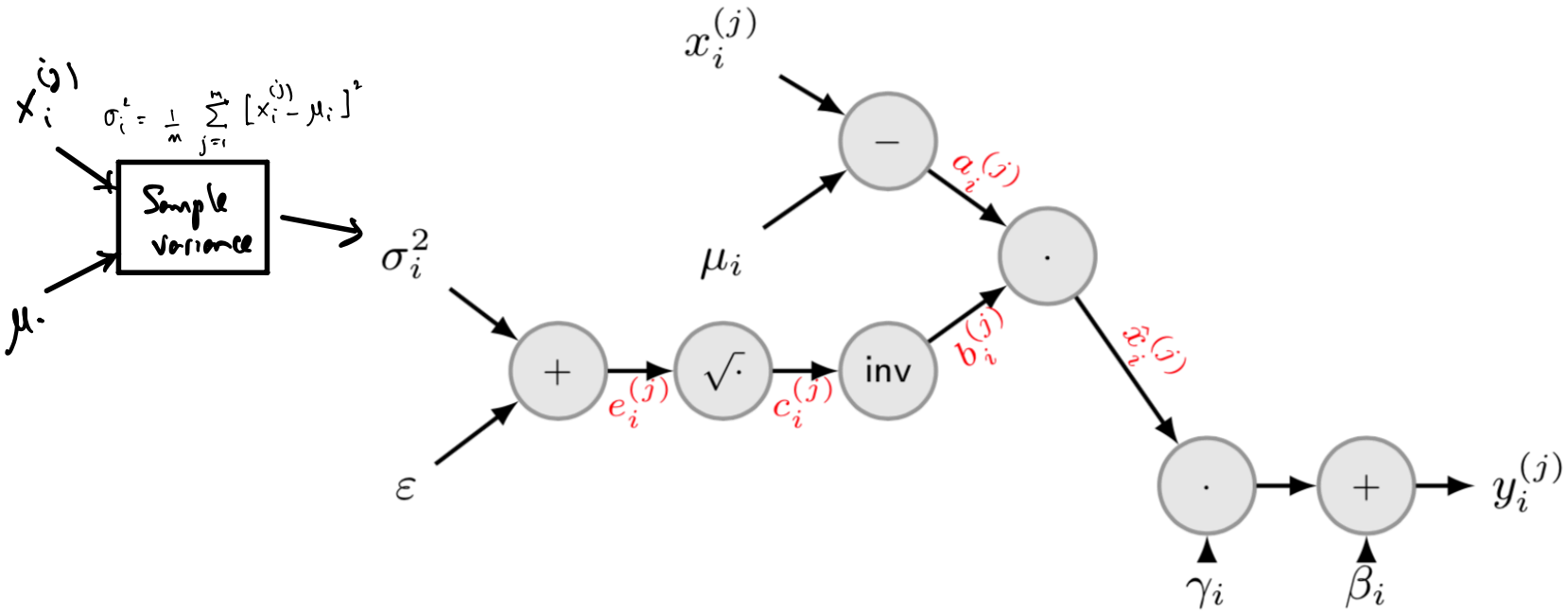
$$\frac{\partial \ell}{\partial \mu} = -\frac{1}{\sqrt{\sigma^2+\epsilon}} \sum_{j=1}^{m} \frac{\partial \ell}{\partial \hat{x}^{(j)}} - \frac{2}{m} \sum_{j=1}^{m} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \sigma^2}$$

$$\frac{\partial \ell}{\partial b^{(j)}} = (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial c^{(j)}} = -\frac{1}{\sigma^2+\varepsilon} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial e^{(j)}} = -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

Batch normalization

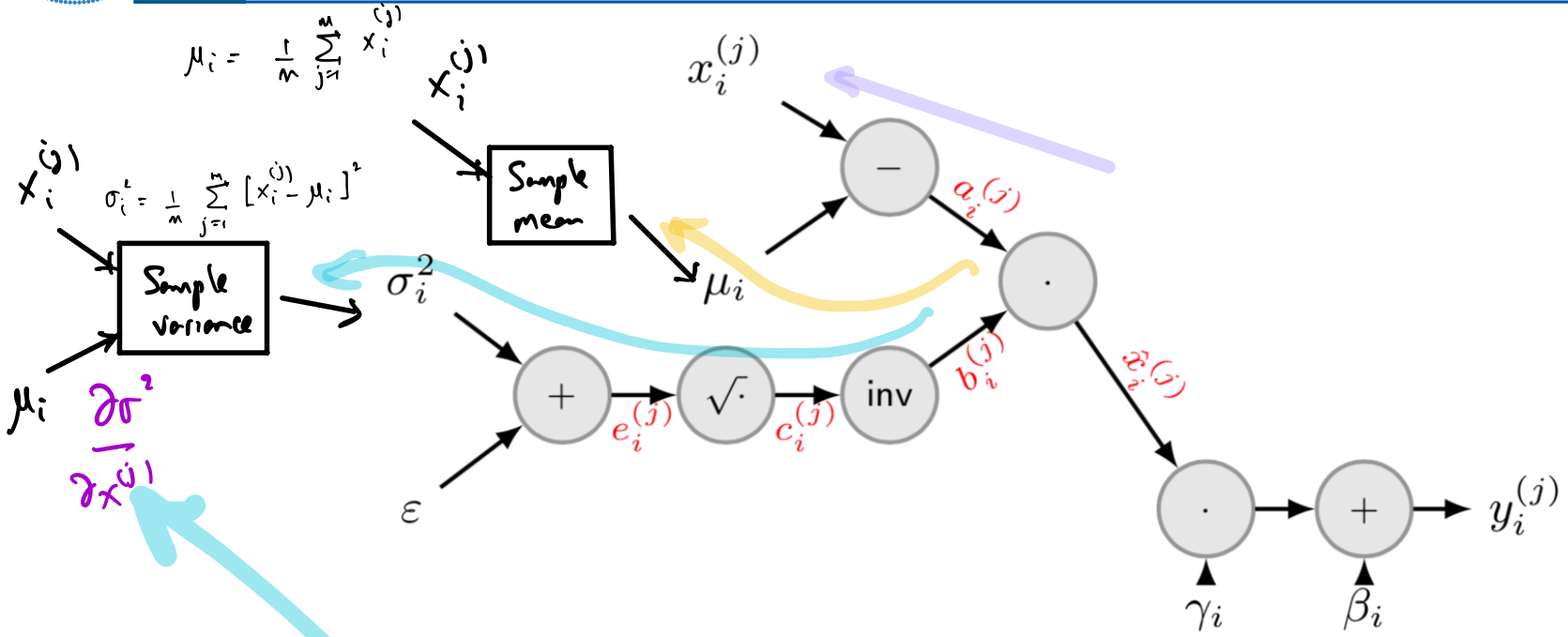


$$\frac{\partial \ell}{\partial a^{(j)}} = \frac{1}{\sqrt{\sigma^2+\epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}} \qquad \frac{\partial \ell}{\partial e^{(j)}} = -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial \sigma^2} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial e^{(j)}}$$

$$= \sum_{j=1}^{m} -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

Batch normalization



$$\frac{\partial \ell}{\partial a^{(j)}} = \frac{1}{\sqrt{\sigma^2+\epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}} \qquad \frac{\partial \ell}{\partial e^{(j)}} = -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

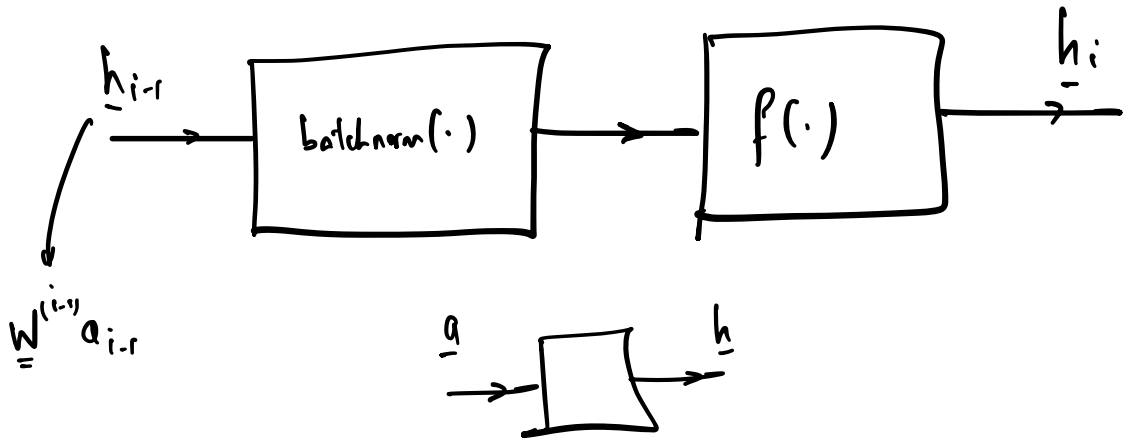
$$\begin{aligned} \frac{\partial \ell}{\partial x^{(j)}} &= \frac{\partial \ell}{\partial a^{(j)}} + \frac{\partial \sigma^2}{\partial x^{(j)}} \frac{\partial \ell}{\partial \sigma^2} + \frac{\partial \mu}{\partial x^{(j)}} \frac{\partial \ell}{\partial \mu} \\ &= \frac{1}{\sqrt{\sigma^2+\epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}} + \frac{2(x^{(j)}-\mu)}{m} \frac{\partial \ell}{\partial \sigma^2} + \frac{1}{m} \frac{\partial \ell}{\partial \mu} \end{aligned}$$

Batch normalization layer

The batch normalization layer is typically placed right before the nonlinear activation. Hence, a layer of a neural network may look like:

$$\mathbf{h}_i = f\left(\text{batch-norm}\left(\mathbf{W}_i \mathbf{h}_{i-1} + \mathbf{b}_i\right)\right)$$

Empirically, batch normalization allows higher learning rates to be used.



TEST TIME

- running average of mean & variance over previously seen inputs
- perhaps use a forgetting factor

$$\mu = \frac{1}{m} \sum_{j=1}^{m} x^{(j)}$$

$$\mu_t = \sum_{j=1}^{t-1} \alpha^{t-j} x^{(j)}$$

What is regularization?

Regularizations

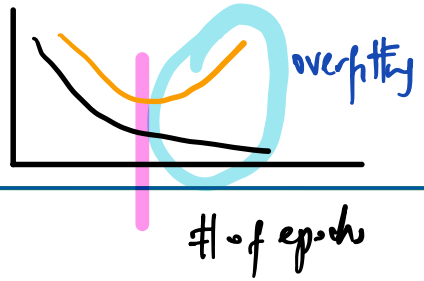
Regularizations are used to improve model generalization. Goodfellow, Bengio, and Courville define regularization in the following way (Deep Learning, p. 221):

> *[Regularization is] any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error.*

In this manner, regularization is used to improve the *generalizability* of the model. Other intuitions:

- Regularization tends to increase the estimator bias while reducing the estimator variance.
- Regularization can be seen as a way to prevent overfitting.
- A common problem is in picking the model size and complexity. It may be appropriate to simply choose a large model that is regularized appropriately.

A simple example of regularization



A simple example: stopping early

One straightforward (and popular) way to regularize is to constantly evaluate the training and validation loss on each training iteration, and return the model with the lowest validation error.

- Requires caching the lowest validation error model.
- Training will stop when after a pre-specified number of iterations, no model has decreased the validation error.
- The number of training steps can be thought of as another hyperparameter.
- Validation set error can be evaluated in parallel to training.
- It doesn't require changing the model or cost function.
- The following is beyond the scope of the class – but in case curious, early stopping can be seen as a form of L^2 regularization (to be discussed in the next slides). See Goodfellow et al., *Deep Learning*, p. 242-5 for an indepth discussion.

What is regularization?

Types of regularization

Regularizations may take on many different types of forms. The following list is not exhaustive, but includes regularizations one may consider.

- It may be appropriate to add a soft constraint on the parameter values in the objective function.
  - To account for prior knowledge (e.g., that the parameters have a bias).
  - To prefer simpler model classes that promote generalization.
  - To make an underdetermined problem determined. (e.g., least squares with indeterminate $\mathbf{X}^T\mathbf{X}$.)
- Dataset augmentation
- Ensemble methods (i.e., essentially combining the output of several models).
- Some training algorithms (e.g., stopping training early, dropout) can be seen as a type of regularization.

Parameter norm penalties

l_2-norm penalty

It is common to associate regularization with parameter norm penalties. These are not specific to neural networks. These are commonly used, e.g., even in linear regression, where specific penalty norms have their own names (e.g., Tikhonov regularization / ridge regression).

You have already seen this in HWs.

Regularization via parameter norm penalties

A common (and simple to implement) type of regularization is to modify the cost function with a *parameter norm penalty*. This penalty is typically denoted as $\Omega(\theta)$ and results in a new cost function of the form:

$$J(\theta; \mathbf{X}, \mathbf{y}) + \alpha\Omega(\theta)$$

regularization function

with $\alpha \geq 0$. A few things to note:

- α is a hyperparameter that weights the contribution of the norm penalty. When $\alpha = 0$, there is no regularization. When $\alpha \to \infty$, the cost function is irrelevant and the model will set the parameters to minimize $\Omega(\theta)$.
- The choice of α can strongly affect generalization performance.

L^2 regularization

A common form of parameter norm regularization is to penalize the size of the weights (L^2 regularization). This is also commonly called "ridge regression" or "Tikhonov regularization." This promotes models with parameters that are closer to 0 (and hence, colloquially speaking, "simpler"). If $\mathbf{w}$ are the model parameters to be regularized, then L^2 regularization sets:

$$\Omega(\theta) = \frac{1}{2}\mathbf{w}^T\mathbf{w}$$

← quadratic l_2 - regularizer

Intuitively, to prevent $\Omega(\theta)$ from getting large, L^2 regularization will cause the weights $\mathbf{w}$ to have small norm.

L^2 regularization (cont)

More formally, when using L^2 regularization, the new cost function is:

$$\tilde{J}(\mathbf{w};\mathbf{X},\mathbf{y}) = J(\mathbf{w};\mathbf{X},\mathbf{y}) + \frac{\alpha}{2}\mathbf{w}^T\mathbf{w}$$

$\|\underline{w}\|_2^2$

l_2- regularization

with corresponding gradient:

$$\nabla_{\mathbf{w}}\tilde{J}(\mathbf{w};\mathbf{X},\mathbf{y}) = \alpha\mathbf{w} + \nabla_{\mathbf{w}}J(\mathbf{w};\mathbf{X},\mathbf{y})$$

$\frac{\partial}{\partial \underline{w}} \frac{\alpha}{2} \underline{w}^T\underline{w} = \alpha \underline{w}$

The gradient step is:

$$\mathbf{w} \leftarrow (1-\epsilon\alpha)\mathbf{w} - \eta\epsilon\nabla_{\mathbf{w}}J(\mathbf{w};\mathbf{X},\mathbf{y})$$

This formalizes the intuition that L^2 regularization will shrink the weights, $\mathbf{w}$, before performing the usual gradient update.

$$\underline{w} \leftarrow \underline{w} - \eta\epsilon\nabla_{\underline{w}}\tilde{J}(\cdot) = \underline{w} - \eta\epsilon\nabla_{\underline{w}}J(\cdot) - \epsilon\alpha\underline{w} = (1-\eta\alpha\epsilon)\underline{w} - \epsilon\nabla_{\underline{w}}J(\cdot)$$

η

Other equivalent statements of L^2 regularization

While we won't discuss these at length in class, it may be worthwhile to work out these equivalences:

- L^2 regularization is equivalent to maximum a-posteriori inference, where the prior on the parameters has a unit Gaussian distribution, i.e.,

$$\mathbf{w} \sim \mathcal{N}(0, \frac{1}{\alpha}\mathbf{I})$$

- When performing L^2 regularization, the component of $\mathbf{w}$ aligned with the ith eigenvector of the Hessian is rescaled by a factor

$$\frac{\lambda_i}{\lambda_i + \alpha}$$

- In linear regression, the least squares solution $\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ becomes:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X} + \alpha\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

scaling the variance of each input feature. The dimensions of $\mathbf{X}^T\mathbf{X}$ that are large (i.e., high variance) aren't affected as much, while those dimensions where $\mathbf{X}^T\mathbf{X}$ is small are.

Extensions of L^2 regularization

Other related forms of regularization include:

- Instead of a soft constraint that $\mathbf{w}$ be small, one may have prior knowledge that $\mathbf{w}$ is close to some value, $\mathbf{b}$. Then, the regularizer may take the form:

$$\Omega(\theta) = \|\mathbf{w} - \mathbf{b}\|_2$$

- One may have prior knowledge that two parameters, $\mathbf{w}^{(1)}$ and $\mathbf{w}^{(2)}$, ought be close to each other. Then, the regularizer may take the form:

$$\Omega(\theta) = \left\|\mathbf{w}^{(1)} - \mathbf{w}^{(2)}\right\|_2$$

L^1 regularization

L^1 regularization defines the parameter norm penalty as:

$$\begin{aligned}\Omega(\theta) &= \|\mathbf{w}\|_1 \\ &= \sum_i |w_i|\end{aligned}$$

FACT: 2.5 decades of optimization to show that it encourages sparse solutions

Intuitively, this penalty also causes the weights to be small. However, because the subgradient of $\|\mathbf{w}\|_1$ is $\text{sign}(\mathbf{w})$, the gradient is the same regardless of the size of $\mathbf{w}$. (Contrast this to L^2 regularization, where the size of $\mathbf{w}$ matters.) Empirically, this typically results in sparse solutions where $w_i = 0$ for several i.

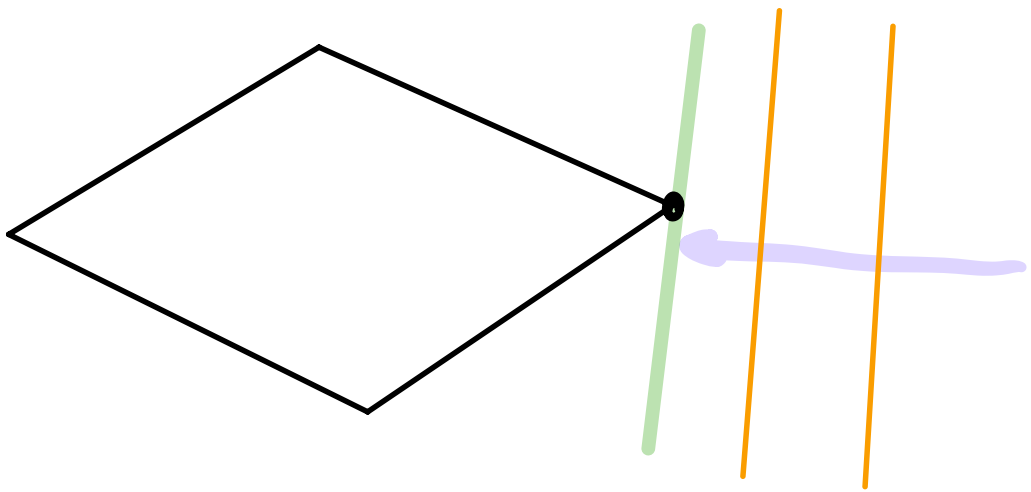
- This may be used for *feature selection*, where features corresponding to zero weights may be discarded.
- L^1 regularization is equivalent to maximum a-posteriori inference where the prior on the parameters has an isotropic Laplace distribution, i.e.,

$$w_i \sim \text{Laplace}\left(0, \frac{1}{\alpha}\right)$$

Sparse representations

Instead of having sparse parameters (i.e., elements of $\mathbf{w}$ being sparse), it may be appropriate to have sparse *representations*. Imagine a hidden layer of activity, $\mathbf{h}^{(i)}$. To achieve a sparse representation, one may set:

$$\Omega(\mathbf{h}^{(i)}) = \left\| \mathbf{h}^{(i)} \right\|_1$$



$\Sigma |w_i|$

Dataset augmentation

encodes invariances

Original image:

Dataset augmentation

Original image:

Flipped image:

Dataset augmentation

Original image:

Flipped image:

Cropped image:

Dataset augmentation

Original image:

Flipped image:

Cropped image:

Adjust brightness

Dataset augmentation

Original image:

Flipped image:

Cropped image:

Adjust brightness

Lens correction

Dataset augmentation

Original image:

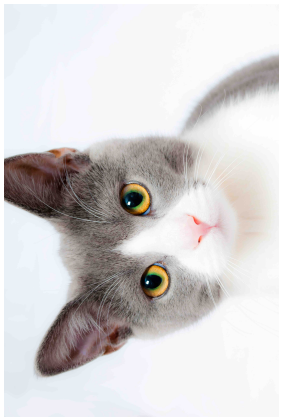
Flipped image:

Cropped image:

Adjust brightness

Lens correction

Rotate



Dataset augmentation

There are various heuristics to keeping the input size the same.

You can sample smaller patches. When resizing or scaling, you can sample a portion of the image but have the minimum dimension still be larger than the patch size.

You can be creative for dataset augmentation.

Encodes invariances into an equivalence class.

Dataset augmentation

Other types of dataset augmentation:

- Inject noise into the network
- Label smoothing

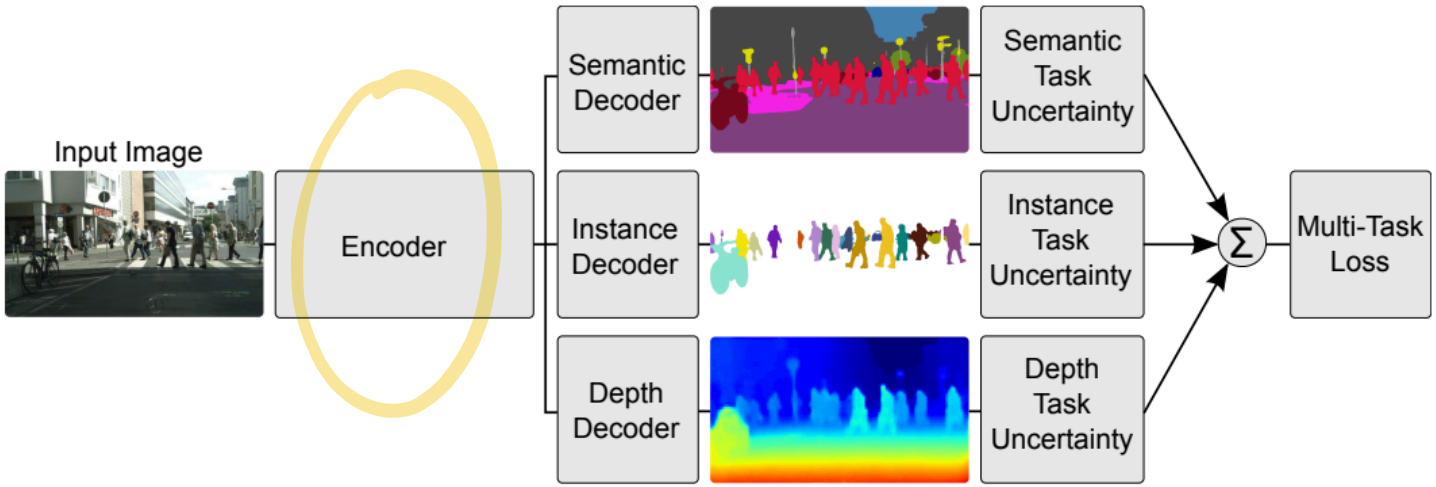
Network	Top-1 Error	Top-5 Error	Cost Bn Ops
GoogLeNet [20]	29%	9.2%	1.5
BN-GoogLeNet	26.8%	-	**1.5**
BN-Inception [7]	25.2%	7.8	2.0
Inception-v2	23.4%	-	3.8
Inception-v2 RMSProp	23.1%	6.3	3.8
Inception-v2 Label Smoothing	22.8%	6.1	3.8
Inception-v2 Factorized 7×7	21.6%	5.8	4.8
Inception-v2 BN-auxiliary	**21.2%**	**5.6%**	4.8

reduction just by dataset augmentation

Szegedy et al., arXiv 2015

Multitask learning

Another way to improve generalization is by having a modeled trained to perform multiple tasks. This represents the belief that multiple tasks share common factors to explain variations in the data.

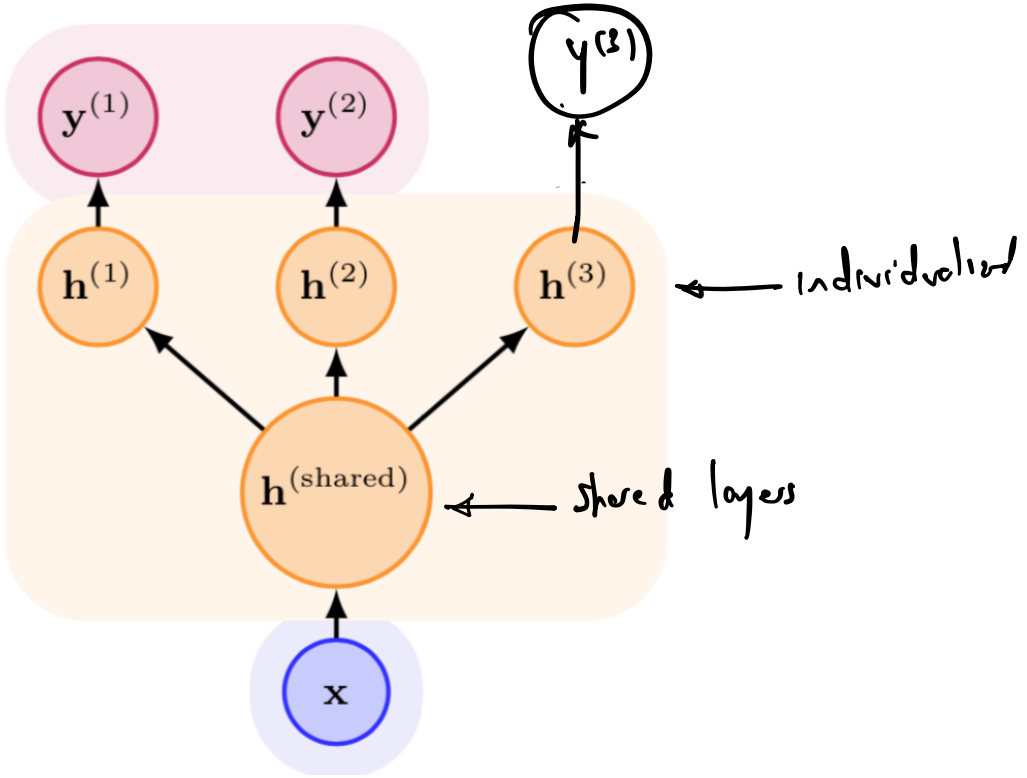


Kendall et al., arXiv 2017

Multitask learning

- The entire model need not be shared across different tasks.
- Here, $\mathbf{h}^{\text{shared}}$ captures common features that are then used by task-specific layers to predict $\mathbf{y}^{(1)}$ and $\mathbf{y}^{(2)}$.
- $\mathbf{h}^{(3)}$ could represent a feature for unsupervised learning.

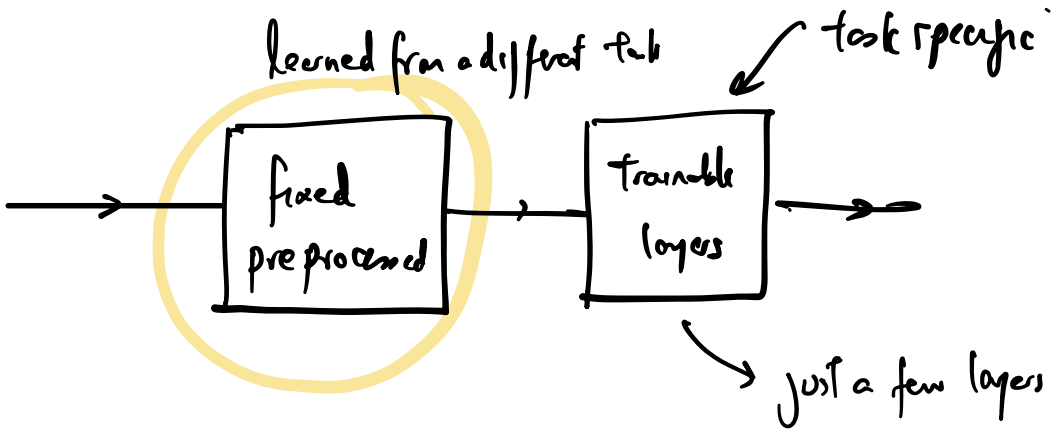
Cumulative error across tasks



Transfer learning

We'll discuss this more in the convolutional neural networks lecture, but a related idea is to take neural networks trained in one context and use them in another with little additional training.

- The idea is that if the tasks are similar enough, then the features at later layers of the network ought to be good features for this new task.
- If little training data is available to you, but the tasks are similar, all you may need to do is train a new linear layer at the output of the pre-trained network.
- If more data is available, it may still be a good idea to use transfer learning, and tune more of the layers.



Ensemble methods

One way to get a boost in performance for very little cognitive work is to use ensemble methods.

The approach is:

1. Train multiple different models
2. Average their results together at test time.

This almost always increases performance by substantial amounts (e.g., a few percentage improvement in testing).

Ensemble methods

- The basic intuition between ensemble methods is that if models are independent, they will usually not all make the same errors on the test set.
- With k independent models, the average model error will decrease by a factor $\frac{1}{k}$. Denoting ϵ_i to be the error of model i on an example, and assuming $\mathbb{E}\epsilon_i = 0$ as well as that the statistics of this error is the same across all models,

$$\begin{aligned}\mathbb{E}\left[\left(\frac{1}{k}\sum_{i=1}^{k}\epsilon_i\right)^2\right] &= \frac{1}{k^2}\sum_{i=1}^{k}\mathbb{E}\epsilon_i^2 \\ &= \frac{1}{k}\mathbb{E}\epsilon_i^2\end{aligned}$$

- If the models are not independent, it can be shown that:

$$\frac{1}{k}\mathbb{E}\epsilon_i^2 + \frac{k-1}{k}\mathbb{E}[\epsilon_i\epsilon_j]$$

which is equal to $\mathbb{E}\epsilon_i^2$ only when the models are perfectly correlated.

One implementation of ensemble methods is via bagging.

$\mathcal{D}$ → create B datasets from $\mathcal{D}$ → sampling with replacement

$\mathcal{D}^{(1)} \ldots \mathcal{D}^{(B)}$

↳ different

Bagging

Bagging stands for bootstrap aggregating. It is an ensemble method for regularization. The procedure is as follows:

- Construct k datasets using the bootstrap (i.e., set a data size, N, and draw with replacement from the original dataset to get N samples; do this k times).
- Train k different models using these k datasets.

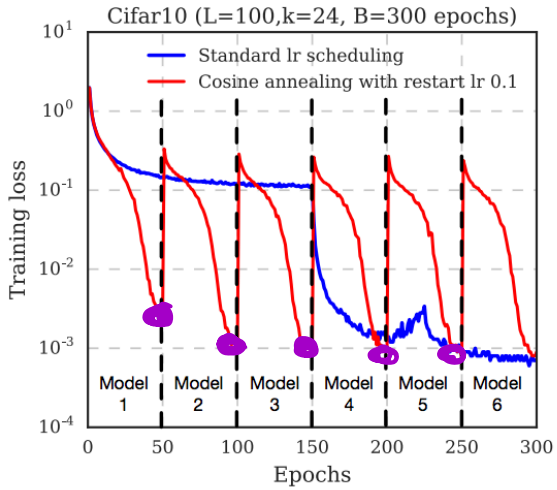
A few notes:

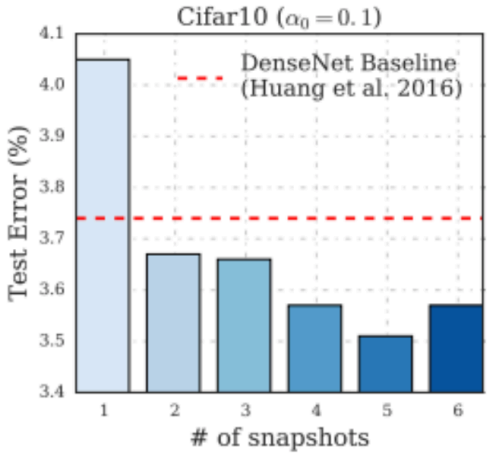
- In practice, neural networks reach a wide variety of solutions given different initializations, hyperparameters, etc., and so in practice even if they are trained from the same dataset, they tend to produce partially independent errors.
- While model averaging is powerful, it is expensive for neural networks, since the time to train models can be very large.

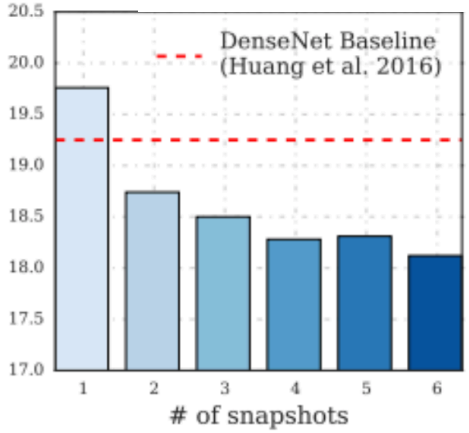
Ensemble methods

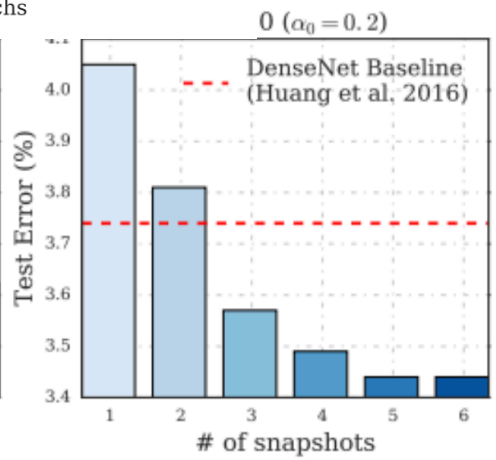
Ways to get around computational expense?

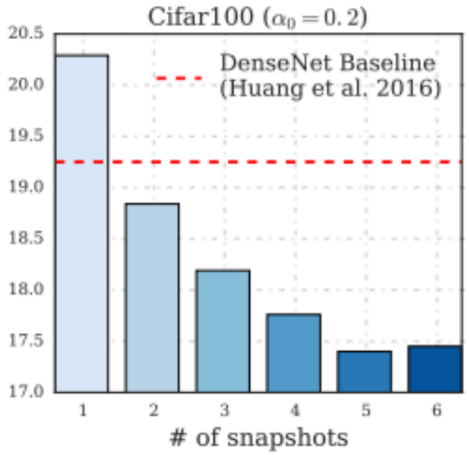
You could take snapshots of the model at different local minima and average them together. See Huang, Li et al., ICLR 2017.







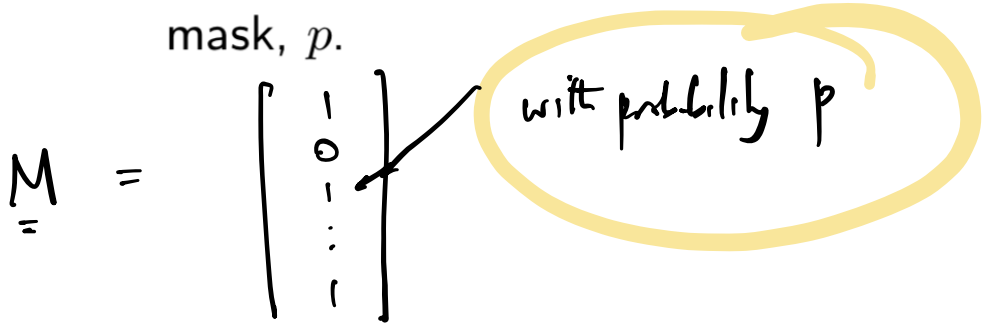




Dropout

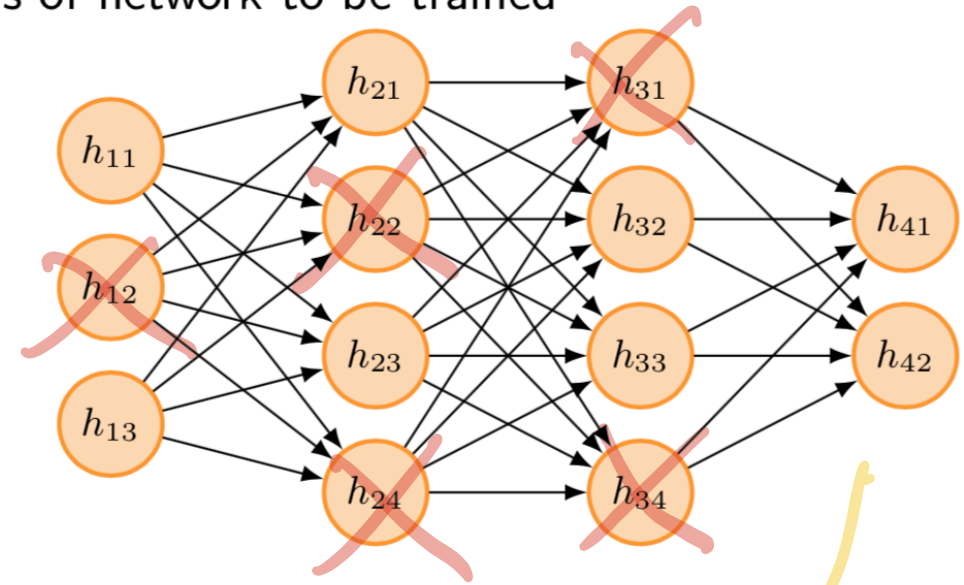
Dropout is a computationally inexpensive yet effective method for generalization. It can be viewed as approximating the bagging procedure for exponentially many models, while only optimizing a single set of parameters. The following steps describe the dropout regularizer:

- On a given training iteration, sample a binary mask (i.e., each element is 0 or 1) for all input and hidden units in the network.
  - The Bernoulli parameter, p, is a hyperparameter.
  - Typical values are that $p = 0.8$ for input units and $p = 0.5$ for hidden units.
- Apply (i.e., multiply) the mask to all units. Then perform the forward pass and backwards pass, and parameter update.
- In *prediction*, multiply each hidden unit by the parameter of its Bernoulli mask, p.



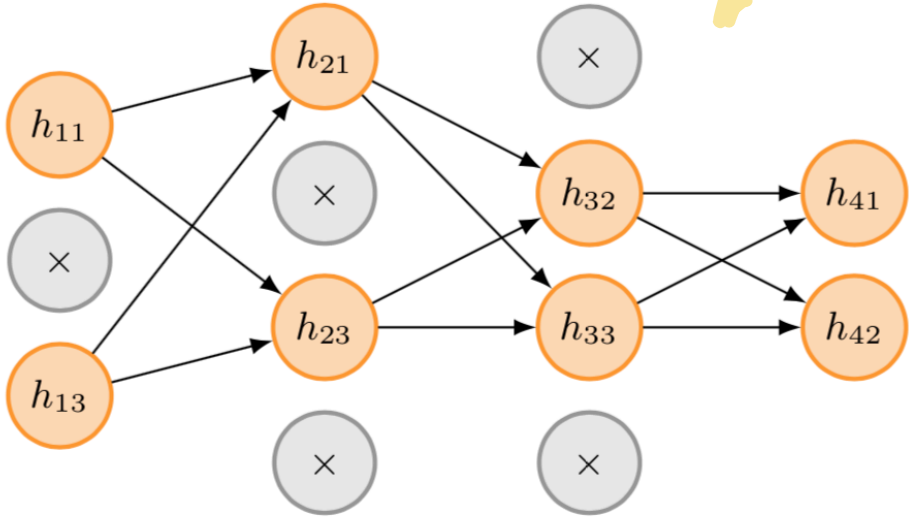
Dropout

Hidden layers of network to be trained



do this independently

Application of random mask on iteration i



Dropout

Dropout in code.

```
p = 0.5 # probability of dropping out
relu = lambda x: x * (x > 0)

def forward(X):
    H1 = relu(np.dot(W1, X) + b1) # First hidden layer activations
    M1 = np.random.rand(*H1.shape) < p # Sample random mask
    H1 *= M1 # Dropout on first hidden layer

    H2 = relu(np.dot(W2, H1) + b2) # Second hidden layer activations
    M2 = np.random.rand(*H2.shape) < p # Sample random mask
    H2 *= M2 # Dropout on first hidden layer

    Z = np.dot(W3, H2) + b3
```

Dropout

How about during test time? What configuration do you use?

$$h_{out} = \text{relu}\left[p \cdot \sum_i w_i h_i\right]$$

$$= p\ \text{relu}\left[\sum_i w_i h_i\right]$$

*We call this approach the **weight scaling inference rule**. There is not yet any theoretical argument for the accuracy of this approximate inference rule in deep nonlinear networks, but empirically it performs very well.*

In this class, instead of scaling the weights, we will scale the activations.

Dropout

```
p = 0.5 # probability of dropping out
relu = lambda x: x * (x > 0)

def test(X):
    H1 = relu(np.dot(W1, X) + b1) * p
    H2 = relu(np.dot(W1, H1) + b1) * p
    Z = np.dot(W3, H2) + b3
```

Note: an additional pro of dropout is that in testing time, there is no additional complexity. With *m* ensemble models, our test time evaluation would scale *O(m)*.

Inverted dropout

A common way to implement dropout is *inverted dropout* where the scaling by $1/p$ is done in training. This causes the output to have the same expected value as if dropout was never been performed.

Thus, testing looks the same irrespective of if we use dropout or not. See code below:

```
p = 0.5 # probability of dropping out
relu = lambda x: x * (x > 0)

def train_forward(X):
    H1 = relu(np.dot(W1, X) + b1) # First hidden layer activations
    M1 = (np.random.rand(*H1.shape) < p) / p # Sample random mask AND normalization by p
    H1 *= M1 # Dropout on first hidden layer

    H2 = relu(np.dot(W2, H1) + b2) # Second hidden layer activations
    M2 = (np.random.rand(*H2.shape) < p) / p # Sample random mask AND normalization by p
    H2 *= M2 # Dropout on first hidden layer

    Z = np.dot(W3, H2) + b3

def test(X):
    H1 = relu(np.dot(W1, X) + b1)
    H2 = relu(np.dot(W1, H1) + b1)
    Z = np.dot(W3, H2) + b3
```

Dropout

How is this a good idea?

1) Dropout approximates bagging, since each mask is like a different model. For a model with N hidden units, there are 2^N different model configurations.

   Each of these configurations must be good at predicting the output.

2) You can think of of dropout as regularizing each hidden unit to work well in many different contexts. → masks

3) Dropout may cause units to encode redundant features (e.g., to detect a cat, there are many things we look for, e.g., it's furry, it has pointy ears, it has a tail, a long body, etc.).

Summary

Here, we've covered tricks that we can do in initialization, regularization, and data augmentation to improve the performance of neural networks.

But what about the optimizer, stochastic gradient descent? Can we improve this for deep learning?

That is our next topic.

Optimization for neural networks

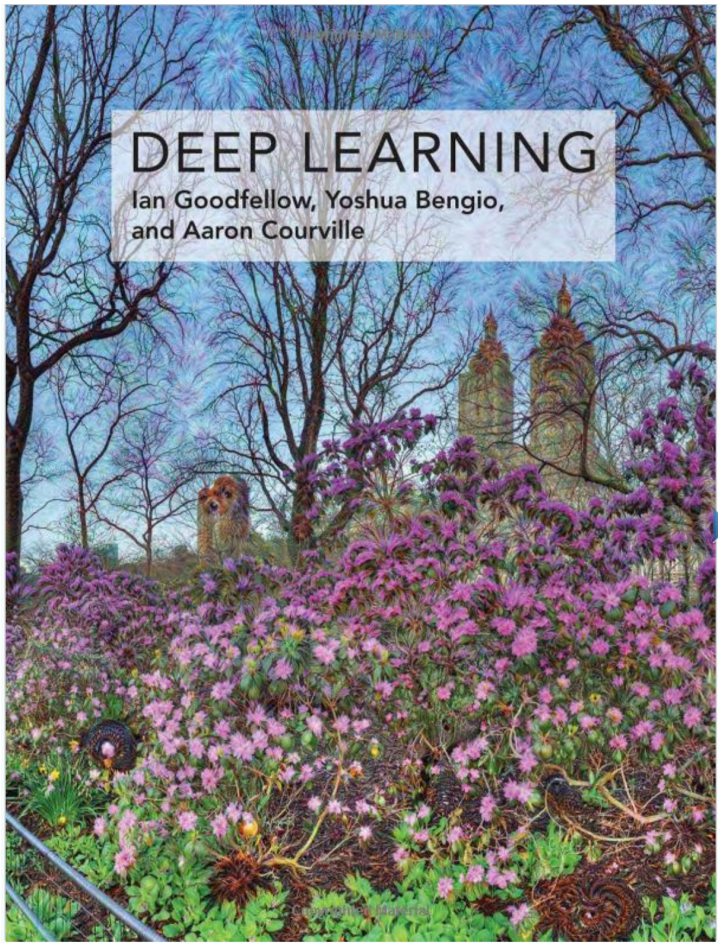
Next we will talk about specific techniques in optimization that aid in training neural networks.

- Stochastic gradient descent
- Momentum and Nesterov momentum
- Adaptive gradients
- RMSProp
- Adaptive moments
- Overview of second order methods
- Challenges of gradient descent

Reading

Reading:

Deep Learning, Chapter 8 (intro), 8.1, 8.2, 8.3, 8.4, 8.5, 8.6 (skim)



Where we are now

At this point, we know:

MLP

- Neural network architectures.
- Hyperparameters and cost functions to use for neural networks.
- How to calculate gradients of the loss w.r.t. all parameters in the neural network.
- How to initialize the weights and regularize the network in ways to improve the training of the network.

We do know how to optimize these networks with stochastic gradient descent. But can it be improved?

In this lecture, we talk about how to make optimization more efficient and effective.

Gradient descent

A refresher on gradient descent.

- Cost function: $J(\theta)$
- Parameters: θ

Then, the gradient descent step is:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

Stochastic gradient descent

Calculating the gradient exactly is expensive, because it requires evaluating the model on all m examples in the dataset. This leads to an important distinction.

- Batch algorithm: uses all m examples in the training set to calculate the gradient.
- Minibatch algorithm: approximates the gradient by calculating it using k training examples, where $m > k > 1$.
- Stochastic algorithm: approximates the gradient by calculating it over one example.

It is typical in deep learning to use minibatch gradient descent. Note that some may also use minibatch and stochastic gradient descent interchangeably.

A note: small batch sizes can be seen to have a regularization effect, perhaps because they introduce to noise to the training process.

Stochastic gradient descent

Stochastic gradient descent proceeds as follows.

Set a learning rate ε and an initial parameter setting θ. Set a minibatch size of m examples. Until the stopping criterion is met:

- Sample m examples from the training set, $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(m)}\}$ and their corresponding outputs $\{\mathbf{y}^{(1)}, \mathbf{y}^{(2)}, \ldots, \mathbf{y}^{(m)}\}$.
- Compute the gradient estimate:

$$\mathbf{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta J(\theta)$$

- Update parameters:

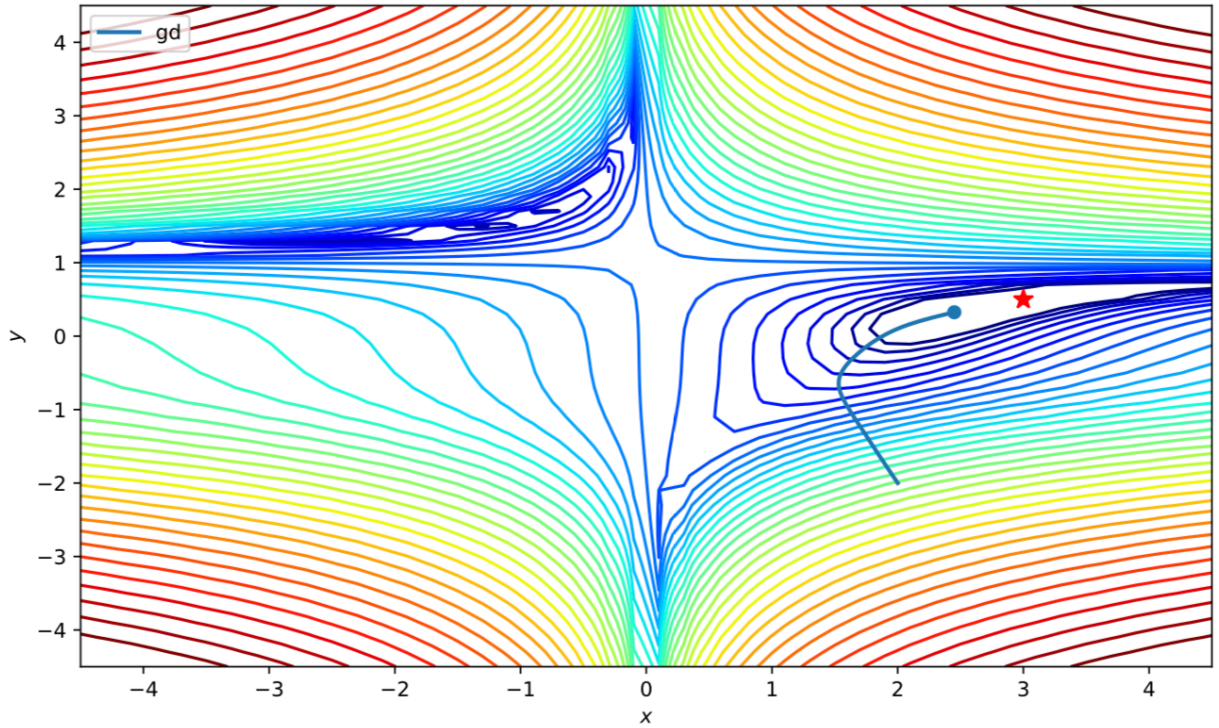
$$\theta \leftarrow \theta - \varepsilon \mathbf{g}$$

Stochastic gradient descent

```
    while last_diff > tol:
        cost, g = func(x)
        x -= eps*g
        last_diff = np.linalg.norm(x - path[-1])
```

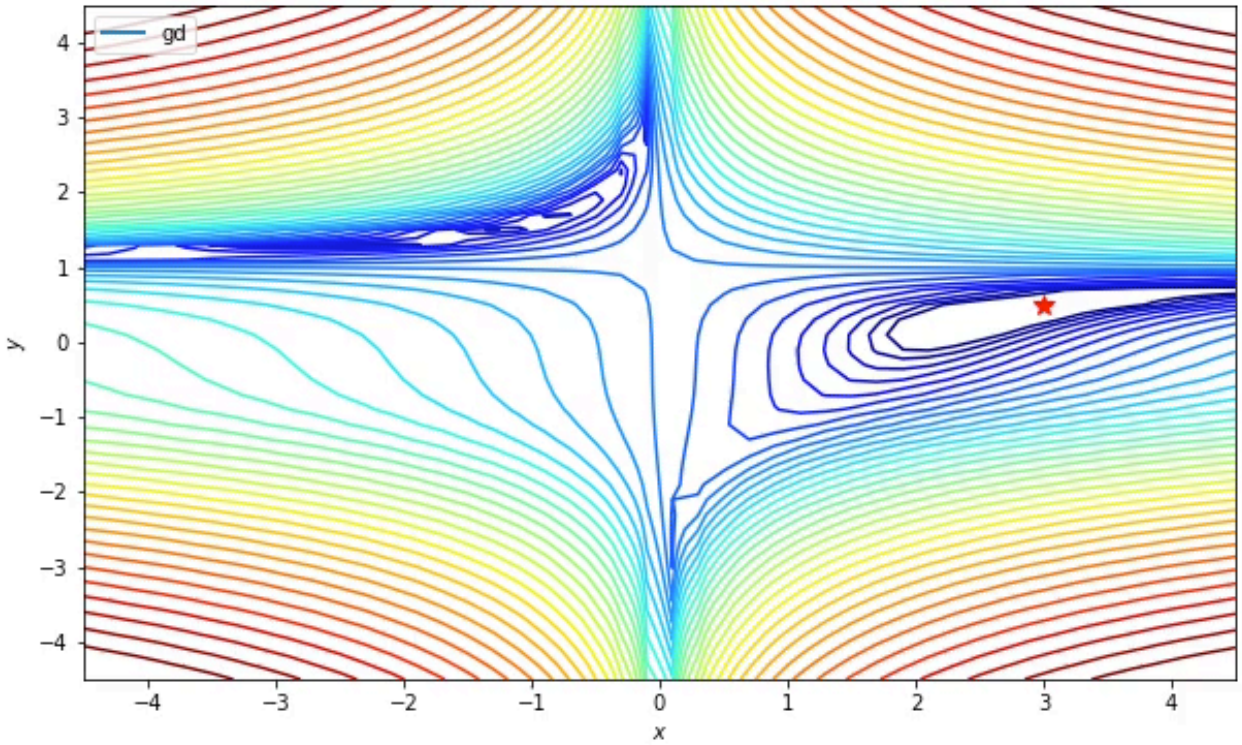
Stochastic gradient descent (opt 1)

The following shows gradient descent applied to Beale's function, for two initializations at $(2, -2)$ and $(-3, 3)$. The two initializations are shown because later on we'll contrast to other techniques. The iteration count is capped at $10,000$ iterations, so gradient descent does not get to the minimum.

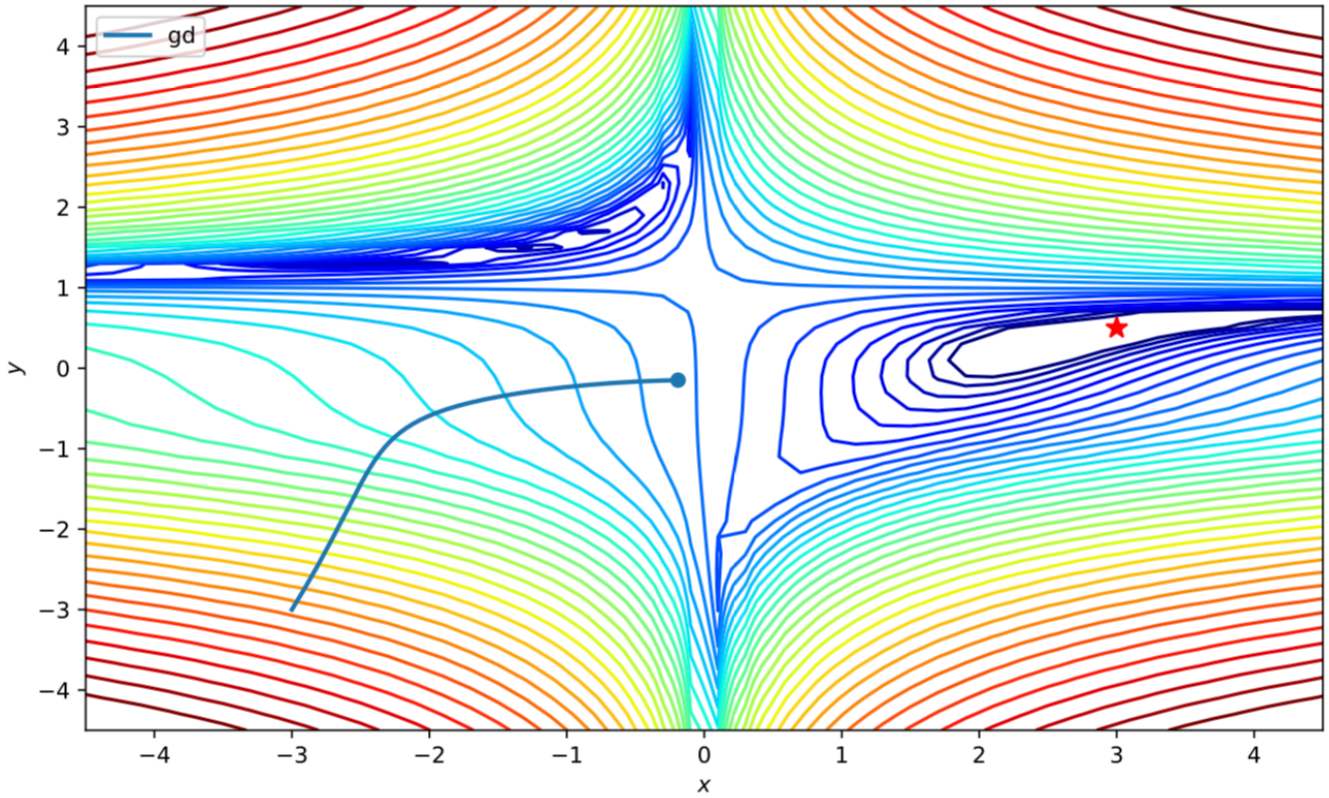


Video: `http://seas.ucla.edu/~kao/opt_anim/1gd.mp4`
Animation help thanks to: `http://louistiao.me/notes/visualizing-and-animating-optimization-algorithms-with-matplotlib/`

Stochastic gradient descent



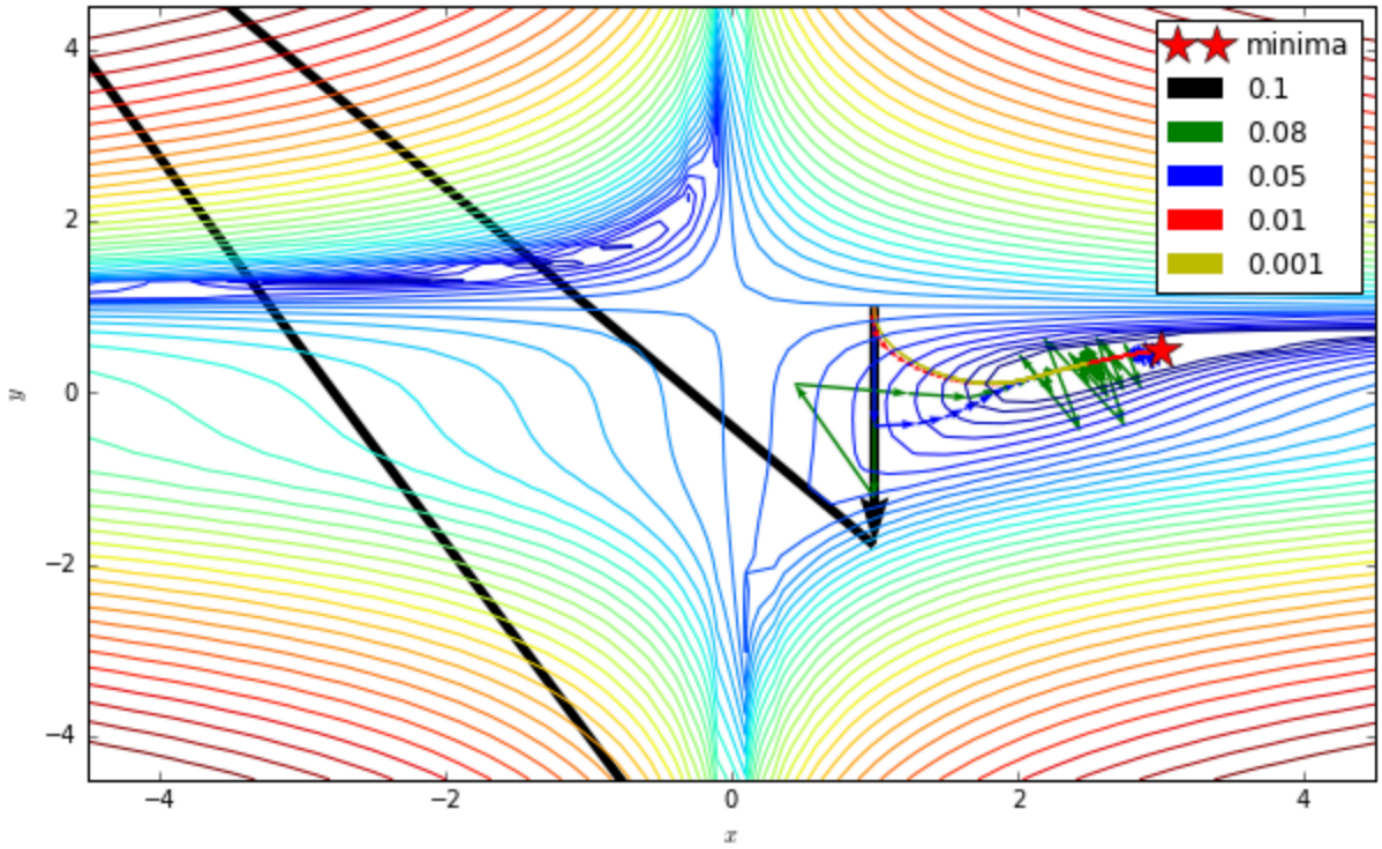
Stochastic gradient descent (opt 2)



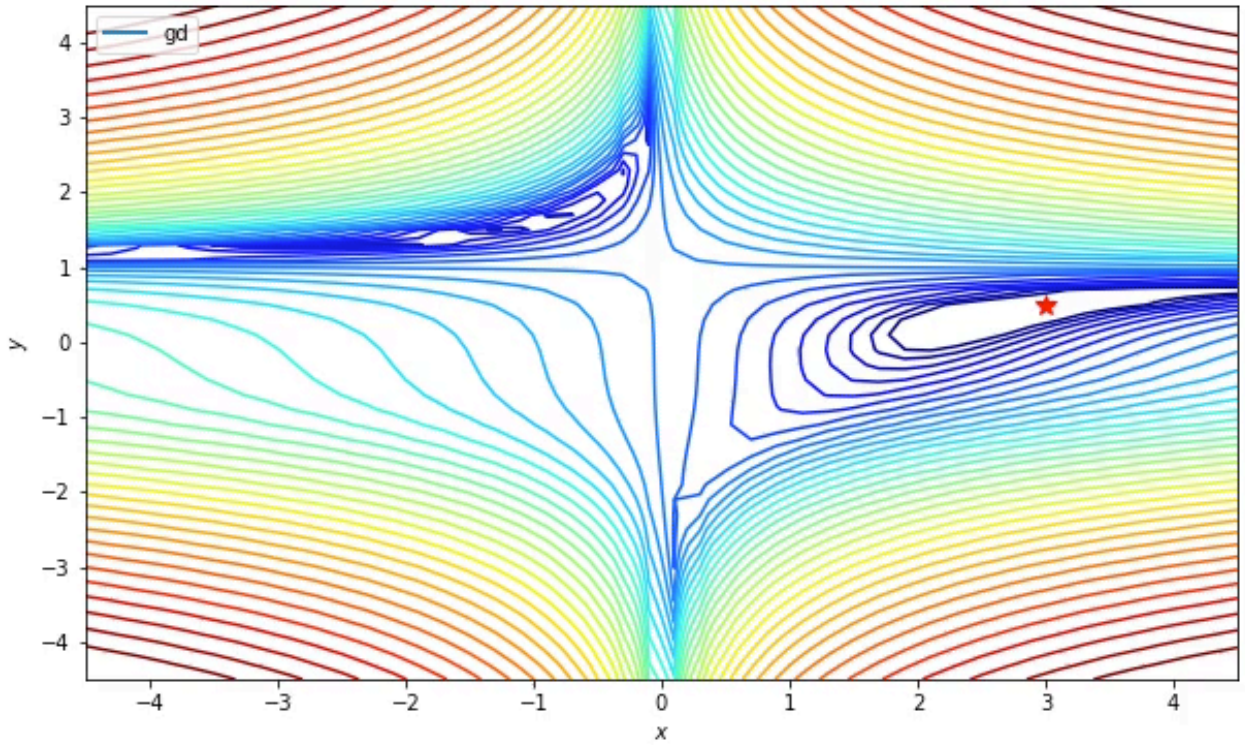
Video: `http://seas.ucla.edu/~kao/opt_anim/2gd.mp4`

Finding the optimal weights through gradient descent

Varying the learning rate:



Stochastic gradient descent



Momentum

In momentum, we maintain the running mean of the gradients, which then updates the parameters.

Initialize $\mathbf{v} = 0$. Set $\alpha \in [0, 1]$. Typical values are $\alpha = 0.9$ or 0.99. Then, until stopping criterion is met:

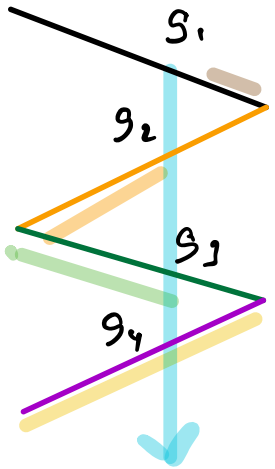
- Compute gradient: $\mathbf{g}$
- Update:

"learning" rate

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \varepsilon \mathbf{g}$$

- Gradient step:

$$\theta \leftarrow \theta + \mathbf{v}$$



$\alpha < 1$

$\underline{v}_0 = 0$

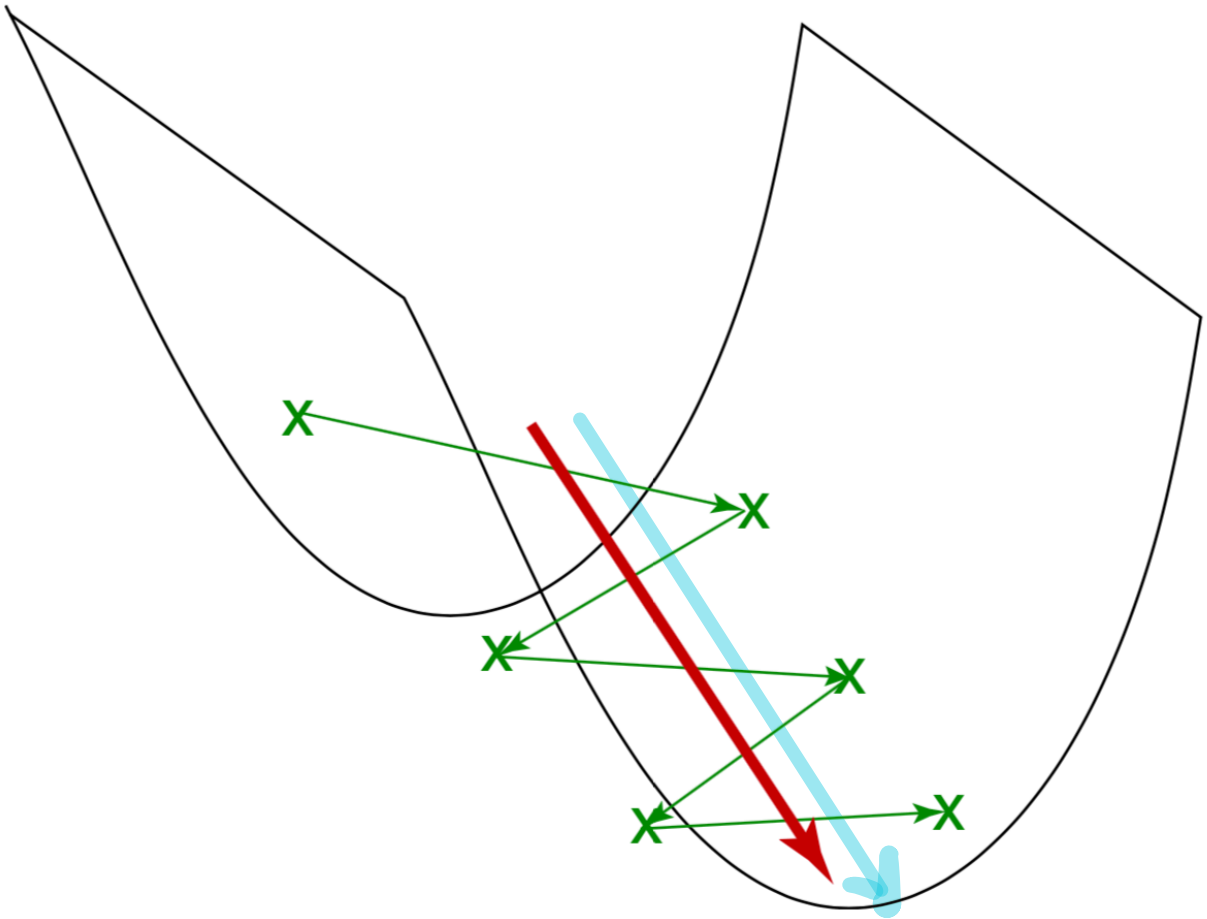
$\underline{v}_1 = -\epsilon \underline{g}_1$

$\underline{v}_2 = \alpha \underline{v}_1 - \epsilon \underline{g}_2 = \alpha(-\epsilon \underline{g}_1) - \epsilon \underline{g}_2$

$\underline{v}_3 = \alpha \underline{v}_2 - \epsilon \underline{g}_3 = \epsilon \left[\alpha^2 \underline{g}_1 + \alpha \underline{g}_2 + \underline{g}_3 \right]$

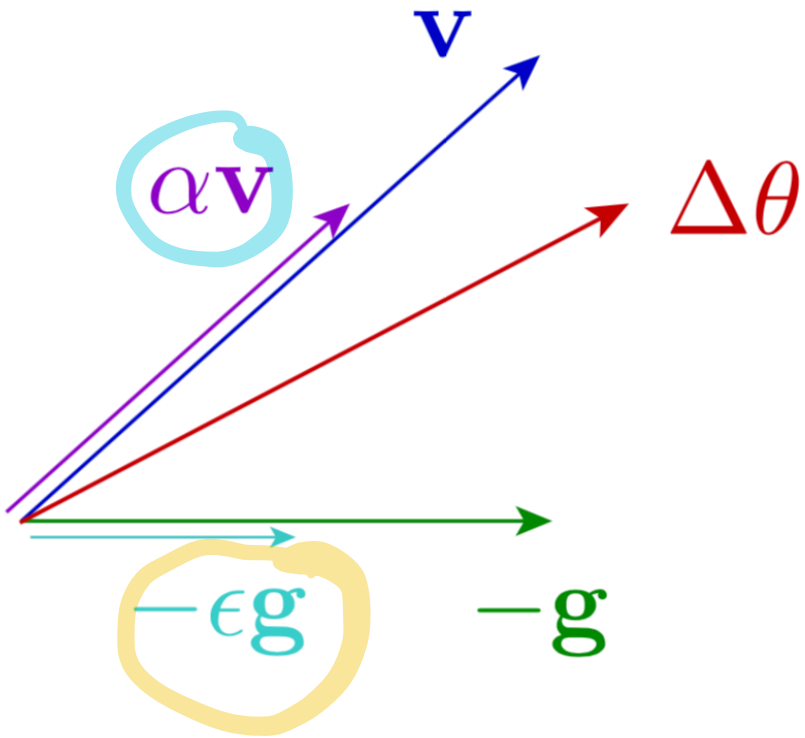
Momentum (cont.)

An example of how momentum is useful is to consider a tilted surface with high curvature. Stochastic gradient descent may make steps that zigzag, although in general it proceeds in the right direction. Momentum will average away the zigzagging components.



Momentum (cont.)

This modification augments the gradient with the running average of previous gradients, which is analogous to a gradient "momentum." The following image is appropriate to have in mind:



$$\underline{v} \leftarrow \alpha \underline{v} - \epsilon \underline{g}$$

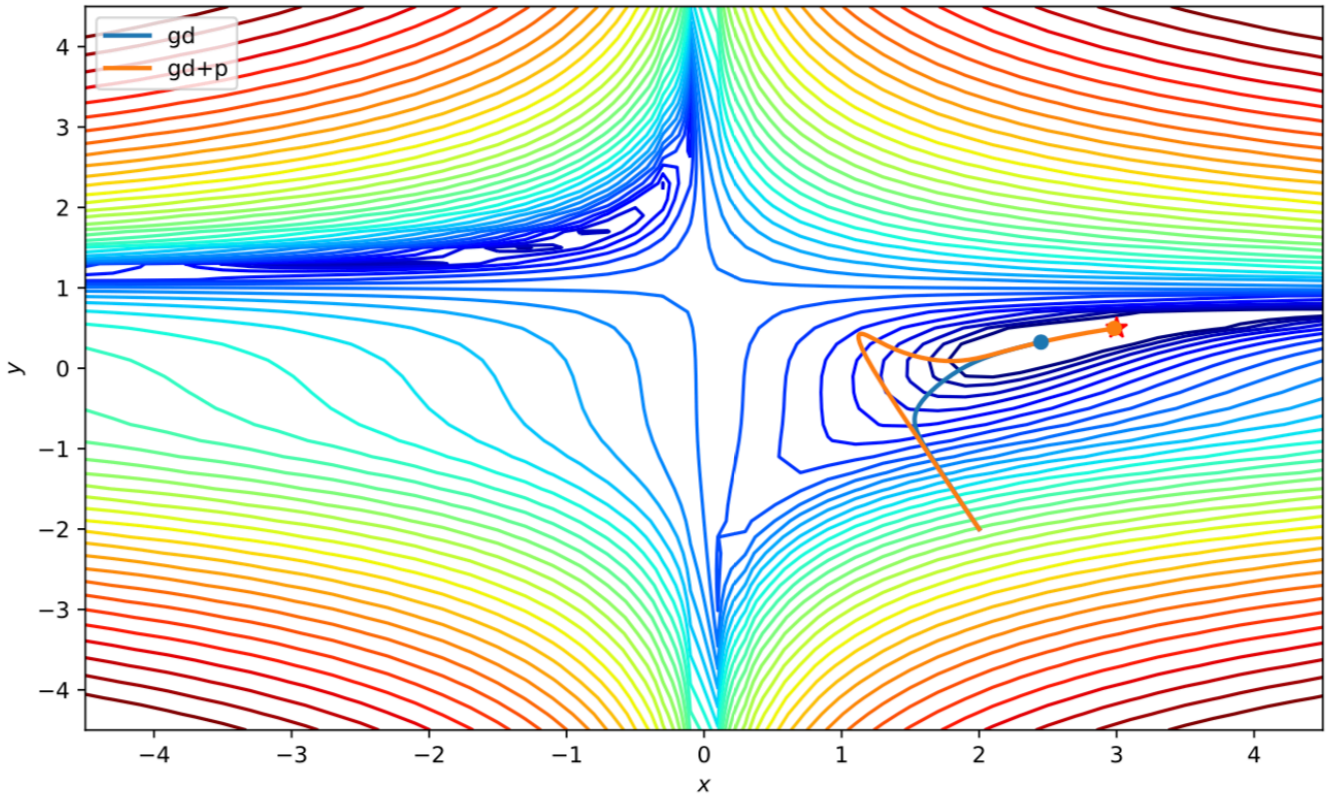
$$\underline{\theta} \leftarrow \underline{\theta} + \underline{v}$$

Momentum

```
alpha = 0.9
p = 0
while last_diff > tol:
    cost, g = func(x)
    p = alpha*p - eps*g
    x += p
    last_diff = np.linalg.norm(x - path[-1])
```

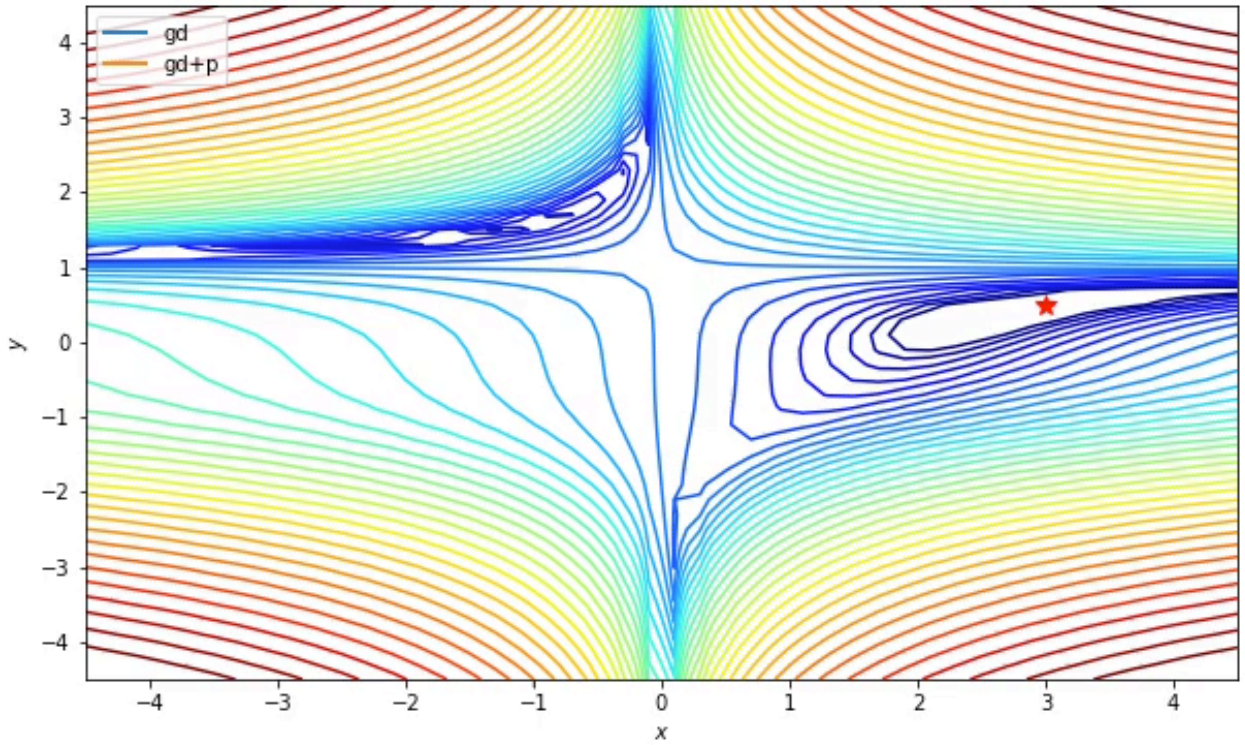
Momentum (opt 1)

gd+p denotes gradient descent with momentum.



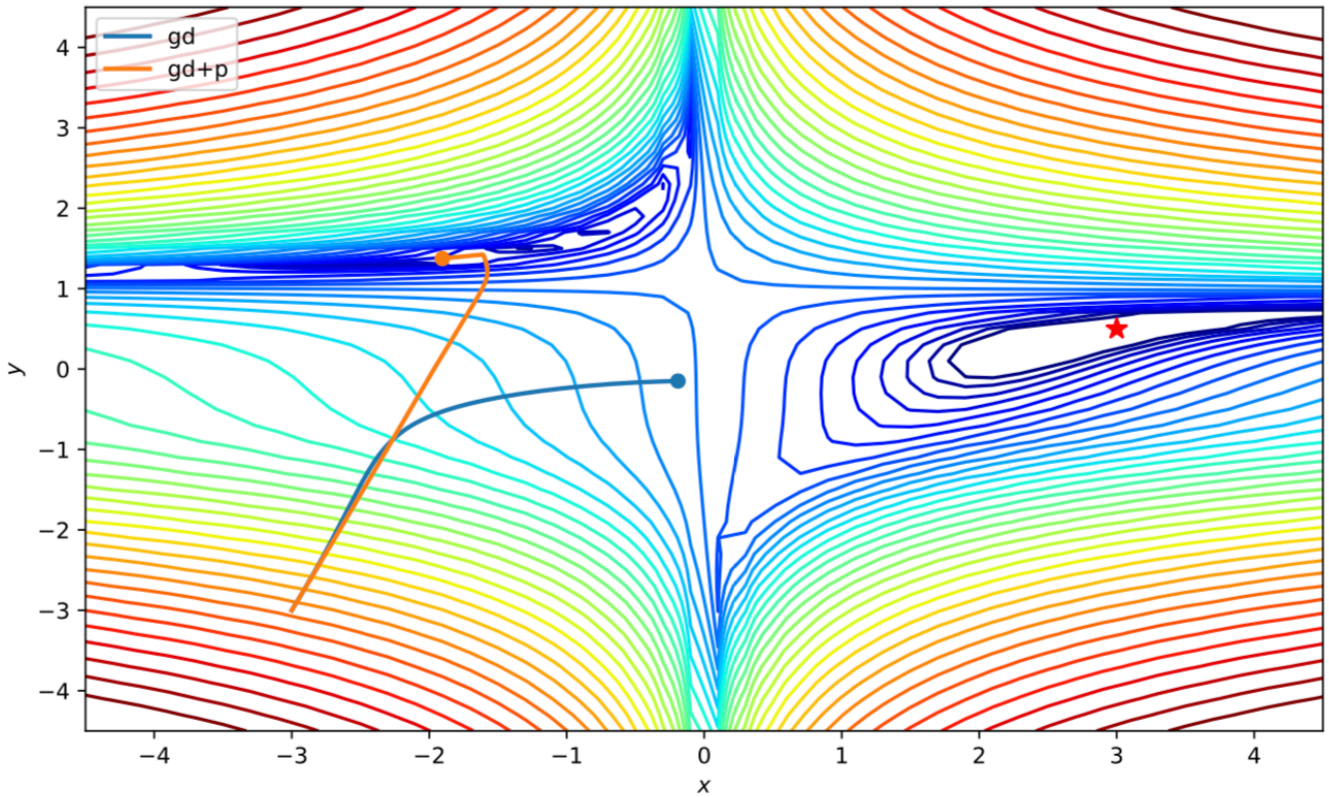
Video: `http://seas.ucla.edu/~kao/opt_anim/1gd_gd+p.mp4`

Momentum



Momentum (opt 2)

Notice how momentum pushes the descent to find a local, but not global, minimum.



Video: `http://seas.ucla.edu/~kao/opt_anim/2gd_gd+p.mp4`

Momentum